

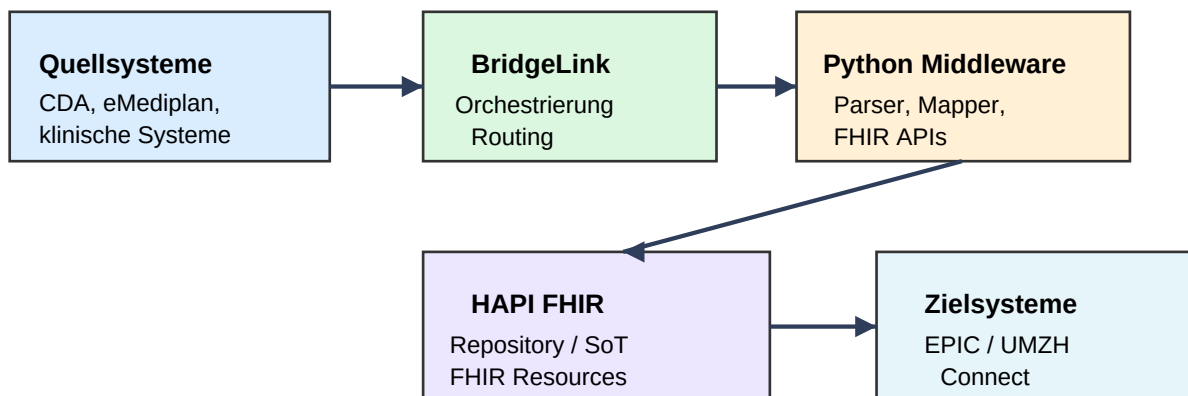
Management Summary

Kubernetes Zielbild fuer den FHIR POC

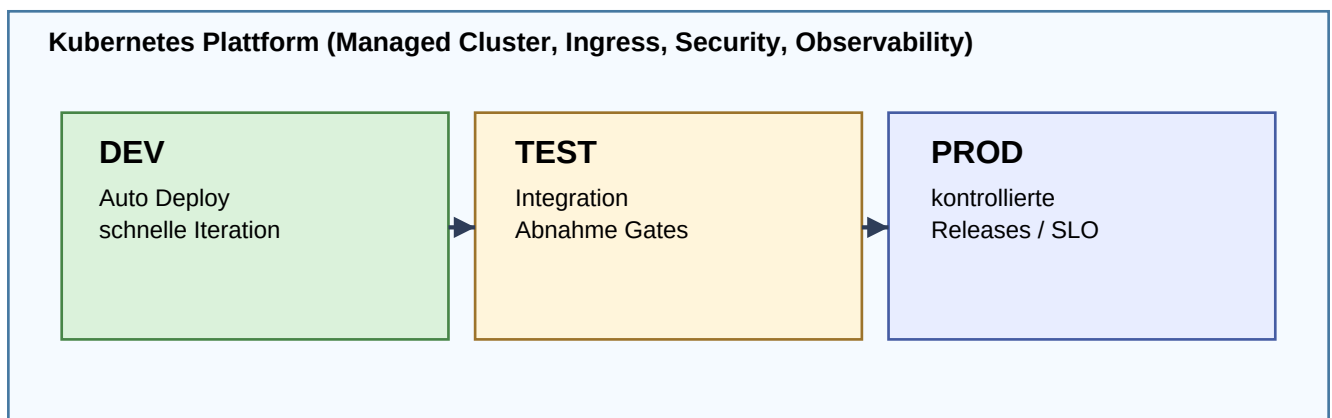
Stand: 2026-07-30



Zielarchitektur



Managed-Service Betriebsrahmen



Management-Kernaussagen

- Managed-Service-First reduziert Betriebsrisiko und beschleunigt Delivery.
- CI/CD mit Quality Gates schafft sichere Promotion von DEV nach TEST nach PROD.
- Kerninfrastruktur: Cluster, Datenbank, Registry, Ingress, Secrets und Monitoring.
- Skalierung erfolgt horizontal in Middleware und resilient fuer PROD mit HA-Pattern.

Detailuebersicht

Stand: 2026-07-30

Der bestehende FHIR POC kann mit vertretbarem Risiko von Docker Compose auf Kubernetes ueberfuehrt werden. Der technische Nutzen liegt nicht in neuer Fachfunktion, sondern in einer professionellen Betriebsplattform mit klaren Umgebungen, reproduzierbaren Deployments, besserer Skalierung und hoeherer Betriebssicherheit. Fuer die GL ist zentral: Der Wechsel lohnt sich vor allem dann, wenn kuenftig mehrere Umgebungen, geregelte Releases und verlasslicher Produktivbetrieb gefordert sind.

- BridgeLink orchestriert und steuert die Verarbeitungsprozesse.
- Die Python Middleware uebernimmt Parsing, Mapping und FHIR Bundle-Erzeugung.
- HAPI FHIR ist das zentrale Repository und die Source of Truth.
- PostgreSQL speichert die FHIR-Daten persistent.
- Nginx stellt den externen Zugriff, TLS und den Reverse Proxy bereit.
- Der Betrieb erfolgt heute ueber Docker Compose und manuelle Betriebsablaeufe.

v

v

Kubernetes Betriebsrahmen:

- DEV: schnelle Iteration und automatisches Deployment
 - TEST: Integrationstests, Abnahme und Regression
 - PROD: kontrollierte Releases, Stabilitaet und Monitoring
-
- Aus Containern werden standardisierte Kubernetes Workloads mit Services, Healthchecks und geregelten Rollouts.
 - Die heutige Docker-Netzwerklogik wird durch interne Service-Kommunikation und DNS im Cluster ersetzt.
 - Reverse Proxy, TLS und Routing werden ueber Ingress und Load Balancer standardisiert.
 - Konfigurationen und Zugaenge werden in ConfigMaps und Secrets verwaltet statt in lokalen Compose-Dateien.
 - Deployments werden ueber CI/CD oder GitOps aus einer Container Registry betrieben.
 - Die Middleware kann horizontal skaliert werden; Datenbank und FHIR-Repository werden sauber als stateful Komponenten abgesichert.
-
- Managed Kubernetes Cluster fuer DEV, TEST und PROD oder mindestens getrennte Namespaces mit klaren Policies.
 - Managed PostgreSQL oder alternativ hochverfuegbarer persistenter Storage fuer den Datenbankbetrieb.
 - Container Registry fuer versionierte und gescannte Images.
 - Ingress Controller, externer Load Balancer, DNS und Zertifikats-Management.
 - Secret Management fuer Passwoerter, Zertifikate und Integrationsschluessel.
 - Zentrales Logging, Monitoring, Alerting und idealerweise Tracing.
 - Backup- und Restore-Prozesse mit nachweisbaren Wiederherstellungstests.
 - CI/CD-Pipeline mit Tests, Security Scans und geregelter Promotion nach DEV, TEST und PROD.
 - Netzwerk-Anbindung zu BridgeLink und angebundenen Quell- und Zielsystemen.
-
- Schnellere und reproduzierbare Releases.

Detailuebersicht (2)

- Bessere Trennung von Entwicklung, Test und Produktion.
 - Hoehere Stabilitaet durch automatische Neustarts, Healthchecks und standardisierte Rollouts.
 - Bessere Skalierbarkeit der Middleware bei steigender Last.
 - Transparenterer Betrieb mit Monitoring, Auditierbarkeit und klaren Verantwortlichkeiten.
-
- Kubernetes bringt zusaetzliche Plattform-Komplexitaet und benoetigt Betriebs-Know-how.
 - Der groesste Infrastruktur-Schwerpunkt liegt nicht bei der Middleware, sondern bei Datenbank, Security und Observability.
 - Ein Eigenbetrieb des Clusters ist fuer ein kleines Team nur eingeschraenkt sinnvoll; empfohlen ist ein Managed-Service-Ansatz.
 - Fuer produktiven Betrieb muessen Betriebsprozesse, Verantwortlichkeiten und Release-Freigaben sauber definiert werden.
-
- Go fuer eine Kubernetes-Zielarchitektur, falls ein stabiler produktiver Betrieb mit mehreren Umgebungen vorgesehen ist.
 - Bevorzugt Managed Services fuer Cluster, Datenbank, Secrets und Monitoring einsetzen.
 - Investitionsfokus auf Plattform-Fundament, CI/CD, Security und Betriebsautomatisierung legen.
 - Umsetzung in drei Phasen: DEV-Fundament, TEST-Promotion, danach PROD-Hardening.